Rldcoin: Interstellar Peer-to-Peer Payments

Runlai Deng

dengrunlai@gmail.com · www.rldcoin.com

19 October 2026

Abstract. A payment system spanning worlds cannot require every local purchase to wait for a distant vote. We describe one native currency, runlai (RLD), with a twenty-one-million lifetime cap and no initial allocation. Each balance or unissued slot has one authoritative home. Local Byzantine consensus excludes conflicting uses; recursive execution receipts establish monetary validity under an authenticated program. Receiving capabilities are issued before future inputs are known. A local preparation delegates a bounded escrow to an independent one-decision controller, which either cancels or releases an export. Historical proving proceeds without stopping the ordinary ledger. The destination then imports or permanently rejects once; only rejection permits recovery after release. Stable placement grants let disjoint public-work claims survive unrelated splits. Explicit policy covers summarize accepted authority families without pretending that old risks disappeared. We specify the state, messages, transitions, proof relation, resource obligations and conditional safety and progress arguments. The construction supports local circulation during isolation and forward payment after preparation, while preserving causal communication delay. Accepted ordering authorities, correct proof implementations, available current data, finite cryptography and sustained service remain assumptions. No signature, proof or supply cap guarantees delivery, perpetual availability or adoption.

1. A payment system for separated worlds

A community should be able to pay a local worker while another world is unreachable. A ship should retain its prepared economy after departure. An authenticated invoice should let the payer choose inputs later, without sending a payment draft across space and waiting for permission to use those particular coins. A valid payment arriving through a succession of contacts should not need another interstellar round trip before its recipient can spend it. These requirements concern both communication and the location of conflict authority.

A globally preferred greatest-work history can eventually discard a competing branch [1]. It cannot also promise that every independently advancing branch of an indefinitely divided economy will remain final. We therefore assign each spendable output one home before use. Distinct homes can continue with distinct funds. Two partitions of the same home cannot both acquire its balance by declaring themselves independent.

The currency is runlai, ticker RLD, for principal, native fees and service escrows. Moving funds creates neither a wrapped currency nor a domain-issued substitute. The lifetime cap is 21,000,000 RLD, with zero spendable genesis supply and zero founder, investor, treasury or validator allocation. Denomination does not establish purchasing power.

The payment claim has two parts. Monetary execution can be independently checked: input ownership, issuance work, amounts, fees and authority transfers follow public rules. Exclusive ordering still needs an accepted conflict authority: a proof that a transaction is valid does not prove that another valid transaction spending the same input was never certified elsewhere. Separating these statements avoids treating either a signature or a validity proof as a solution to every fault.

This is peer-to-peer payment in the sense that owners authorize exact payments, any willing party may carry evidence, and there is no compulsory central payment intermediary or continuously reachable Earth directory. Local settlement uses explicitly admitted validators. Automatic trust in arbitrary new committees and universal permissionless consensus are not supplied by the term peer-to-peer.

2. Model, identities and assumptions

A domain is an authenticated replicated state machine, not a geographical monopoly. Several domains may coexist on a planet, and a ship can form its own domain. A stable descriptor binds the common currency identity C_RLD, initial validators, consensus and encoding rules, cryptographic suites, execution-proof profiles, separate ordinary-signing and controller-key roots, resource bounds, fee schedules and authorized succession. A self-asserted location or relay address is not that descriptor.

Identifiers are constructed before the objects that sign them. A stable domain ID hashes an initial identity statement containing an identity public key and nonce, not its later configuration, key trees or its own digest. The monetary descriptor names D_0 by that identity and fixes the initial logical slot intervals; C_RLD excludes subsequent certificates and state roots computed from C_RLD. Initial configuration attestations and key-tree roots bind those already fixed identifiers. A recipient's accepted anchor authenticates the complete initial descriptor; a second configuration claiming the same identity is not silently adopted. This ordering avoids circular genesis, key-tree and certificate commitments.

The baseline local consensus is PBFT with n = 3f + 1 distinct admitted validators, the prepare, commit, view-change and new-view rules of [2], and at least 2f + 1 distinct commit messages for a finalized sequence position. Messages additionally bind C_RLD, the stable domain, configuration, sequence, view, exact operation-batch digest, predecessor state and checkpoint header H defined in Section 4.1. Ordinary owner operations are deterministic PBFT commands. A signed state root without the protocol's commit conditions is not finality. A different consensus suite requires a separately identified profile and authenticated adoption, not a local reinterpretation of a certificate.

Every relevant configuration assumes at most f Byzantine validators, correct deterministic validation, authentic keys, collision-resistant hashes and durable non-rollback consensus state. Safety does not require a timely stellar response. Liveness requires a responsive local quorum, eventual communication conditions sufficient for PBFT, fair bounded admission and funded resources. These are obligations of the accepted domain, not consequences of a radio connection.

An owner may control several keys, carriers may duplicate or lose messages, and an adversary may reorder contacts, lie about routes, submit conflicting spends or withhold evidence indefinitely. Owners need not be online simultaneously. A minority partition waits. A single surviving validator has no authority to replace a failed quorum; a deliberately single-validator domain has no Byzantine replication tolerance.

Receiving policies select acceptable configuration histories and maximum exposures before accepting funds. An available descriptor with identical rules is not automatically a trusted ordering authority. Threshold compromise, including applicable historical-key compromise, remains outside the safety claim. A finite cryptographic horizon, sound execution proofs for the specified monetary program, available current state and an eventual physical path are prerequisites of the corresponding verification and delivery claims.

3. Money with one authoritative home

One RLD equals 10^24 atomic runlai units. Amounts are nonnegative bounded integers; no floating-point rounding participates in authorization. C_RLD commits to one authenticated monetary genesis, the denomination, the complete issuance budget, work rules, retirement rules and succession restrictions. Matching a name or ticker is insufficient. A rule change cannot create a second supply under another identifier and call both balances the same RLD.

A spendable output binds its identifier, C_RLD, owner policy, home and a normalized map from acceptance-cover ID to integer amount. A cover is an immutable, explicitly accepted set of authority families and required cryptographic/custody rules. Execution proves that every actual dependency falls within the advertised cover. Components with the same cover coalesce; changing to a broader cover requires explicit owner authorization. Different covers remain separately allocated. These are acceptance descriptions of RLD, not domain currencies. Their amounts sum to the output amount. Section 9 defines the information and risk tradeoff.

The local state contains ordinary outputs, prepared-object delegations E, permanent destination decisions J, receiving reservations R, source capability-use decisions S, issuance segments A, controller-key allocation counters, cancellation/recovery redemption maps, accepted cover definitions, configuration history and service escrows. All maps

are committed by the executed state. No public observation of a remote balance is signing authority. Every operation has a typed action tag, exact predecessor and unique identifier. Payment, capability, issuance and service-object namespaces are distinct.

3.1. A fixed, divisible issuance budget

The cap is C = 21,000,000 RLD = 21 x 10^30 atomic units. There are N = 4096 budget roots with L = 1,000,000 slot identities each. Slot (s,j), 0 <= s < N and 0 <= j < L, pays rho = 0.005126953125 RLD = 5126953125000000000 atomic units once. Thus N L rho = C exactly. Every Mint reward goes to its signed work author. Genesis creates zero spendable RLD and no founder, investor, treasury or validator allocation. There is no presale, halving, tail emission, reserve withdrawal or replacement issuance for loss or retirement.

A placement grant G binds root s, original bounds [lo,hi), home, placement ancestry and creation statement. Its ID is immutable while its slots remain at that home. A local ordered segment index represents the still-active portions of G. A segment (G,l,h,k,mint_anchor) has l <= k < h and remaining slots [k,h). Segment boundaries are an indexing device, not the identity to which placement work is bound.

A public claim selects [a,b), 1 <= b-a <= 256, wholly within a currently unconsumed segment of G. Preparation removes exactly those slots into its controller and retains nonempty [k,a) and [b,h) at the source under the SAME G. It never closes the grant for unrelated slots. The left segment preserves its existing Mint anchor; the newly opened right frontier receives a tagged anchor derived from the split body. Already consumed slots stay consumed.

Installation at another home or cancellation/rejection return creates a new grant for precisely the detached slots. It does not restore their old grant or merge them into a source segment. Source residuals retain their original G. The 4096 roots therefore do not limit issuance homes: arbitrary many disjoint installed segments can exist, bounded by the finite slot space. No domain gains budget by declaring its existence.

3.2. Public placement rather than discretionary awards

Initially root grants [0,L) reside in the explicitly accepted bootstrap domain D_0. Its quorum has no administrative award or direct reserve-payment command. Honest validators process public ClaimRange rather than a private beneficiary list. Initial quorum selection, fair admission, hardware advantage and availability remain trust and economic boundaries.

A typed ReceiveCapability for RLD_ISSUANCE_INTERVAL names the source, destination, accepted roots and covers, maximum slot count, controller/proof suites and funded resources. It excludes the future grant, selected range, source head and Mint cursor. Placement work then binds C_RLD, G, [a,b), destination, capability digest and claimant policy. It excludes segment boundaries, source height and mutable Mint anchors. Validation checks that the complete selected range still belongs to G and is unconsumed here.

For example, splitting [900,901) from a grant containing [100,120) leaves the latter's G and work unchanged. Mint below a leaves the claim unchanged too. A claim loses only if an operation actually consumes/delegates one of its selected slots, changes their placement authority, closes its capability, or otherwise violates its signed terms. Two disjoint ranges commute as slot allocations even when their shared index segment is split. Native consensus still orders the operations; no simultaneous overlapping exercise is allowed.

A successful PrepareClaim verifies proportional placement work and capability-use authorization, detaches the exact slots, marks S[p] = Delegated(x), and creates a controller with kind issuance, range, destination and claimant cancellation policy. It issues no RLD. The proof/controller mechanism in Sections 4 and 5 applies to this typed object. Controller Exported permits destination installation or permanent rejection. Controller Canceled permits one source return without involving the distant destination. After Exported, only certified destination rejection permits return. Every return preserves slot identities and cover ancestry under a new grant.

Grant IDs hash their typed creation bodies, excluding resulting IDs and roots. Segment opening anchors hash G, exact frontier and transition role, excluding resulting anchors. Reusing an old work solution at a new home or after return fails because G changes for those slots. An unrelated split cannot change G. Counters and permanent decisions never reset on configuration succession.

Claims need no prior RLD balance and pay no Mint reward; sponsors or service agreements fund preparation resources. The 256-slot maximum bounds one claim, not a human entitlement. Proportional work discourages cheap acquisition of a large range but cannot prove equitable distribution or useful placement. Admission limits controller count, verification work and retained segment bytes before preparation. Permanently unavailable grants/controllers reduce attainable issuance; no timeout recreates slots.

3.3. Public work and Mint

Mint and ClaimRange use Argon2id version 0x13 [3], m = 65536 KiB, t = 3, p = 1, output 32 bytes, empty secret and associated data. Enc is the deterministic encoding in Appendix A. Each author signs the complete candidate. Work binds its payout or claimant policy, so copying a solution cannot redirect it.

```
z_mint = enc("RLD-MINT",protocol_id,C_RLD,G,j,home,mint_anchor,
             payout_policy,initial_cover,nonce)
z_claim = enc("RLD-CLAIM",protocol_id,C_RLD,G,a,b,home,destination,
              capability_digest,claimant_policy,cover,nonce)
z = z_mint or z_claim for the typed action
P = SHA256(enc("RLD-WORK-PASSWORD",protocol_id,C_RLD,z))
S = first16(SHA256(enc("RLD-WORK-SALT",protocol_id,C_RLD,z)))
y = Argon2id(P,S;m=65536,t=3,p=1,v=0x13,out=32)
weight = 1 for Mint; b-a for ClaimRange
T_base = 2^240 - 1
valid_work = big_endian_integer(y) <= floor(T_base/weight)
```

Mint requires j to be the first unconsumed slot of its current segment and its exact opening anchor. It consumes that slot once, pays rho, advances the cursor and derives the next anchor from the exact Mint body. Neither G nor other segments' anchors change. Splitting above an already open frontier preserves that frontier and its work; a newly opened right frontier receives its own anchor. Work never binds mutable segment bounds.

Under a uniform-result model, Mint succeeds with probability 2^-16 per evaluation, or 65,536 evaluations on average. An n-slot claim succeeds with probability (floor(T_base/n)+1)/2^256. Claim work earns no reward and replaces no later Mint work. These are work counts, not a clock, ASIC-resistance result or ordinary-CPU profitability promise. There is no retarget or guaranteed exhaustion date; hardware progress and unavailable ranges affect actual issuance.

Invalid-candidate verification also costs resources. Each domain uses finite queues for work, payments and controller decisions, with a declared positive scheduling share for each admitted nonempty class. Per-contact challenge/rate limits and funded sponsors bound work admission; a final ticket digest receives one retained result rather than repeated expensive verification. Authentication alone grants no unlimited queue. These local anti-abuse rules do not create a per-person fair lottery. Liveness assumes a bounded admitted workload; an adversary exceeding it can cause refusal, never weaker work or copied slots.

If the work primitive loses confidence, affected Mint stops. An authenticated replacement must preserve every consumed/delegated identity and prevent old/new double exercise. A local vote cannot change rho or replenish budget.

4. Local settlement and its meaning

Owners authorize exact currency, inputs, recipient/change/fee cover allocations, nonce, operation kind and versions. Native replicas execute every deterministic command before PBFT voting. Ordinary settlement requires its native QC, exact output inclusion and valid execution, either through maintained fully verified state or a certified execution receipt. A submitted request or unsigned wallet reservation is not settlement.

An ordinary local output can be spent in the next valid local command. No stellar reply, mining maturity or departure proof is required. A domain needs locally homed RLD and a responsive accepted quorum, not an issuance grant, to circulate money. Local performance depends on computation, communication, durable writes and admission.

Reference execution starts at the common zero-issuance genesis and authenticated descriptors. It checks ownership, slot work and partition, covers, amounts, fees, capability delegation, terminal decisions, redemptions and succession.

Recursive receipts summarize the same relation. A source native head is never held still merely to prove a future export: native preparation first commits a fixed historical object, and that object has its own terminal controller.

4.1. Execution receipts and certified checkpoints

A native checkpoint H binds C_RLD, profile, domain, sequence, predecessor certified checkpoint, previous/resulting state roots, exact batch digest, active signing configuration and material-manifest root. State includes every monetary, issuance, delegation, decision, redemption and service map. The manifest excludes its own header, receipt and QC. An execution receipt publishes exactly H; the current QC is checked separately and is not an input to its own candidate transition.

A receipt for an already COMMITTED checkpoint can be generated later. It recursively verifies its certified predecessor and executes every intervening committed batch, including handoffs, in order. It remains a statement about that exact historical checkpoint even after thousands of later native payments. No current-head equality is required to finish the old proof. Fresh native commands cannot consume a prepared object's delegated principal or slots.

```
NativeExec(H; witness):
  authenticate genesis, or predecessor receipt AND predecessor QC
  reconstruct exact authenticated state; execute each batch in order
  verify owners, work, covers, remote objects and all exclusions
  require computed roots, configuration and manifest match H
  publish H; resolve every recursive assumption
Certified(H) = valid unconditional receipt(H) AND matching QC(H)
```

Prepare creates a unique one-decision controller x with immutable initial statement I_x: currency, source/capability, kind, exact ticket, amount or slots, cover allocation, cancellation/recovery policy, controller configuration and material bounds. Its source creation proof is Certified(H_p) plus exact inclusion of I_x and S[p] = Delegated(x). The controller has an independent PBFT namespace and only two terminal values: Exported or Canceled. It has no ordinary balance-transfer command and cannot change its initial principal or destination.

Controller replicas use the same n = 3f + 1 / 2f + 1 PBFT rules, with messages additionally binding x, I_x, terminal value, view and phase. The initial state has no earlier decision. Each terminal proof executes one transition from I_x and includes the source creation proof. The Exported relation checks the exact owner-authorized ticket, valid capability and complete current material; Canceled checks the bound cancellation authorization. Release readiness is also a replica rule: the Exported receipt is generated for the fixed candidate, verified and retained before its QC is released. Canceled needs no prior recursive receipt: replicas directly execute its bounded authorization/return rule and can issue its terminal QC immediately after durable commitment. Its own proof bytes/QC are never recursive inputs to that candidate. A current terminal QC signs the exact resulting header; proof bytes and its own QC are excluded from that header.

```
ControllerExec(I_x,decision; witness):
  verify Certified(H_p) and exact creation/delegation inclusion
  require initial controller state and pinned controller keys
  if Exported: verify exact ticket, capability and ready material
  if Canceled: verify bound cancellation authorization and return
  require one legal terminal transition and exact public journal
  resolve every recursive assumption
```

A controller's Exported proof may be prepared while its Canceled request competes. Only a matching terminal QC grants authority; an uncommitted proof has no effect. PBFT locking excludes both terminal outcomes under the fault bound. Each controller can wait for its own proof without freezing the native ledger or another controller. Native preparation can continue up to its funded outstanding-object limit. Fair queues and a finite admitted proving workload are progress premises, not claims of unlimited throughput.

4.2. A pinned recursive proof profile

The reference realization uses RISC Zero zkVM 3.0.6 SuccinctReceipt STARK recursion [17,18]. The monetary descriptor authenticates the exact verifier/control root, hash parameters, native and controller program image IDs, successful-exit requirement, journal encoding and limits. Those IDs are implementation commitments computed from actual programs; a paper does not invent their hashes or initialize a currency. Both programs must implement the relations given here and in Appendix A.

A native guest verifies the previous native receipt and QC, each imported controller/native receipt and exact source binding, then executes the complete batch. A controller guest verifies its source creation receipt/QC and inclusion, then its one terminal transition. A native cancellation redemption can directly execute that same bounded Canceled transition from its locally verified I_x and matching terminal QC; this is defined native execution, not accepting an unchecked remote balance. Its resulting native receipt proves those checks for a later cold receiver.

All recursive assumptions must be resolved with real subordinate receipts. The outer verifier checks successful unconditional execution, exact approved program/control/hash parameters and journal, then the corresponding QC and membership data. Fake/development, unsuccessful, unsupported or unresolved receipts refuse. Seal integrity alone is insufficient. Work, signatures, cover lifting and permanent-decision checks occur in the authenticated program, not an unchecked host assertion.

Correct guest code and backend soundness are additional premises. Recursion does not prove its own program correct, make Argon2 cheap, supply missing state or exclude an unseen valid quorum fork. The construction identifies the exact relation an implementation must realize; practical proving/verification time, memory and independent security need implementation evidence. Loss of a proof's private scratch transcript can be repaired from retained exact witnesses; loss of all such data can halt proof production.

4.3. Validity, current data and ordering

A cold recipient receives the controller terminal receipt/QC, creation binding, exact ticket/capability inclusion, full current output table, relevant immutable cover definitions, current authority/key descriptors and bounded authenticated access material. Creation dependencies may be recursively folded into the terminal receipt, but its journal binds their exact identities. The recipient durably installs enough current material to execute and prove its next payment without asking an unreachable source. A receipt with missing current data remains pending.

Historical validity proofs reject inflation or invalid ownership under soundness. Exclusive ordering remains PBFT's separate premise. Two valid programs can execute conflicting spends on two branches certified by a compromised quorum. Known incompatible certificates trigger an incident; remote silence cannot reveal a hidden branch. No proof or collateral removes that information limit.

Prepared principal is locally unavailable to ordinary spending but cancelable through its controller while Exported has not finalized. Released principal is in flight and has no source timeout refund. These distinct wallet states must not be called the same finality. A responsive local quorum settles other available balances while a controller proves, waits or cancels. Authority loss, rollback, unsound proofs and unavailable data remain explicit failures.

5. Export: surrender before sending

The destination preissues a finite ReceiveCapability family. Each leaf p binds C_{RLD}, exactly one stable source domain, destination/recipient, object kind, principal or slot limit, accepted cover IDs, source/controller/destination cryptographic profiles, fee and recovery formulas, current-material/proof bounds and capacity funding. A use key, or an explicitly public-use policy, authorizes consuming that leaf. Actual monetary owners are selected later and separately authorize the entire ticket.

A family ID hashes currency, source, destination, identity and nonce, excluding its later leaf root. The finalized family statement binds one leaf root and N funded leaf obligations; p = (family_ID,index). Membership, native execution receipt, QC and reservation inclusion authenticate the coupon. Merkle compression removes repeated publication bytes,

not the N liabilities. Wildcard multi-source leaves are forbidden: independent sources could otherwise spend one capacity slot twice.

Source preparation allocates x from C_RLD, source identity, creator policy and a fresh owner nonce; I_x and the ticket then bind x, avoiding circular IDs. It verifies the capability's complete certified reservation, owner/use signatures, inputs, cover terms, fees, controller-key allocation and available custody before committing. For payment, exact inputs become change, explicit source-service fees and locked principal a = recipient + destination_fee. For issuance, exact selected slots leave the active source segments. The native operation records I_x and S[p] = Delegated(x). Neither a solved work hash nor an uncommitted preparation consumes anything.

```
Prepare(T,p):
  require certified funded coupon, use authorization, S[p] unused
  require all monetary/work authors authorize exact T and covers
  require funded controller/custody capacity and fresh key index
  consume exact inputs or detach exact still-active selected slots
  create authorized change/fees and controller escrow I_x
  set S[p]=Delegated(x); retain immutable creation and material
```

The source's ordinary ledger then continues. Any retained-witness prover can build the historical creation receipt. Controller Exported is eligible only when its complete compliant capsule and terminal execution receipt are ready; its own quorum finalizes and durably releases the matching QC. The controller uses only I_x, so unrelated native heads do not invalidate that proof. No new stellar approval is required when an unprepared payer changes inputs.

Before Exported, the recovery policy may authorize controller Canceled. Its QC permanently excludes release and permits one native RedeemCancel at the original source. That operation restores the locked payment minus only its agreed cancellation fee, or the exact detached slots under a new grant; it marks the redemption and source coupon Closed. Cancellation consumes that coupon, not another payment's receiving capacity. Its authenticated closure can later free the destination's unused resource obligation. Cancellation cannot erase already paid preparation services or guarantee success after Exported wins.

An entirely unused coupon closes only with use-policy authorization and certified native CloseCapability while S[p] is unused. A Delegated coupon closes through controller Canceled, never by native timeout or administrator override. Configuration changes retain its exact controller binding. Lost use keys or an unavailable source/controller may strand capacity, so family size and outstanding preparation are finite and funded.

After Exported, x carries its terminal proof/QC and exact current material forward. An optional native observation can annotate that outcome, but destination import needs no further source acknowledgment. The source can never reclaim via Canceled once export has won. A returning invoice/coupon is still required for an unknown source or depleted inventory; prepared compatible payments need no draft-specific stellar round trip.

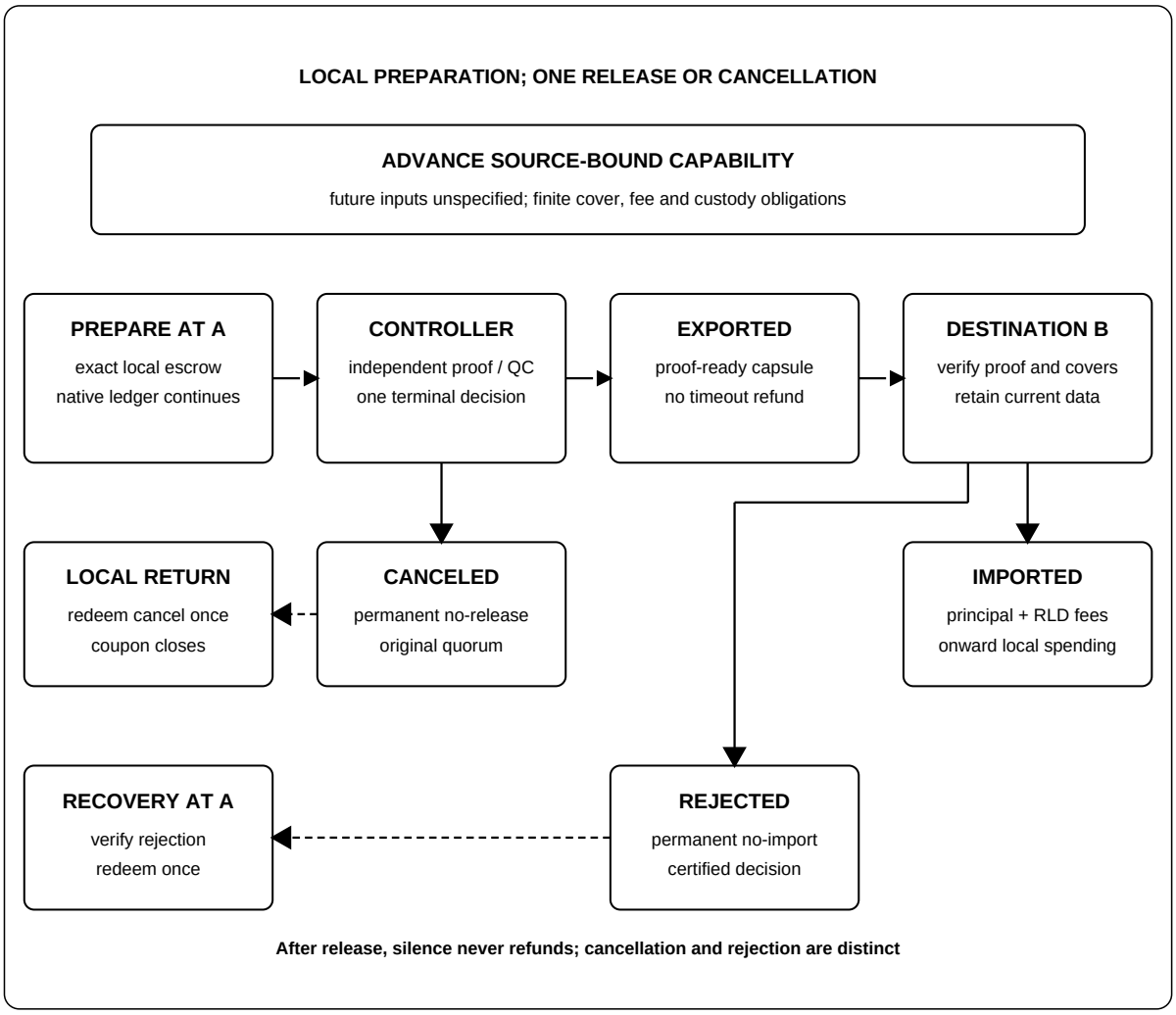


Figure 1. A native preparation creates an independent one-decision controller. Cancellation returns only unexported escrow; after release, destination rejection is the sole recovery path. Ordinary local settlement continues while proving.

6. Import: a single destination decision

Any carrier can deliver the exact ticket, coupon family/leaf evidence, controller Exported receipt and matching QC, source creation binding, cover material and declared current data. The receiver verifies the pinned programs and every journal binding, then the controller keys authenticated by native preparation. A self-signed controller is not a new trusted source. The source capability-use map must show Delegated(x) at creation; the controller's terminal certificate proves its effective Exported state.

```
Import(x,capsule):
  verify certified native creation and unique S[p]=Delegated(x)
  verify controller Exported execution receipt and matching QC
  require exact source/destination/currency/ticket/leaf identities
  require R[p]=Reserved, J[x] absent and no known conflict
  verify every advertised cover and all original limits/fees
  retain complete current material durably
  atomically set R[p]=Imported(x),J[x]=Imported(ticket_digest)
  create exact recipient and destination-service RLD outputs
```

A compact terminal receipt may recursively discharge creation proof verification; its public journal and carried current descriptors still identify the exact source, preparation and controller. Membership alone never proves money. Every output cover must include the destination's authority family and satisfy the coupon. A needed cover broadening is part of the owner's signed T; the destination cannot invent it after arrival.

Installation of an issuance object uses the same exclusion but creates only the exact new grant, never an RLD output. Import and terminal-map consumption are atomic. Duplicates return the retained result; changed bytes, mixed identities and incomplete data refuse or remain pending. A conforming outstanding coupon survives ordinary fee/policy changes. Authenticated compromise or broken cryptography is an incident, not an import obligation for known conflicting money.

After import, RLD is local and can circulate or enter another prepared controller without a reply to the old source. Carriage through C adds no monetary authority; import into C and export again is a distinct transfer. Stable source, controller, coupon and decision namespaces survive all authenticated compaction and succession.

7. Rejection, cancellation and stranded value

Controller Canceled and destination Rejected are different decisions. Canceled certifies that the object was never released and can return locally. Rejected applies only to an already released, fully authenticated export and permanently excludes its import. The destination serializes Imported and Rejected against both R[p] and J[x]. A recipient or bound recovery owner can authorize a rejection request; it is not guaranteed to beat import.

A source RedeemCancel verifies its own retained I_x, the exact controller Canceled QC and cancellation authorization by directly executing that one transition. It checks the permanent cancellation-redemption map before restoring principal or slots. No recursive proving delay is required for an available source already holding the authenticated creation. A cold source must reconstruct that creation or verify its receipt; an unsupported shortcut is forbidden. Later native proofs include the complete cancellation check.

After controller Exported, source recovery instead requires a destination execution receipt/QC proving permanent Rejected for the exact x and p. It also checks the controller's valid Exported proof, native creation and source recovery-redemption map. It restores a-r plus the authorized recovery-fee output r, 0 <= r <= a, exactly once. Issuance recovery restores the same slots under a new grant. A late native annotation of Exported cannot reopen a redeemed object.

Silence cannot distinguish a never-imported ticket from an imported ticket whose reply was lost. No elapsed-time refund is allowed after release. Invalid packets, queue refusal and expired transport envelopes are not certified rejection. Rerouting needs rejection, recovery and a new owner-authorized payment; a carrier cannot change destination.

A responsive local controller permits canceling a preparation that cannot obtain a suitable proof; no source monetary head must freeze for that attempt. Loss of its quorum/keys or current material can still strand the local escrow. Loss of the destination quorum can strand released value. A replacement administrator cannot assume old authority. These are availability failures of the chosen fault model, not occasions to recreate money.

Optional insurance spends already issued RLD under explicit premium, finite exposure and adjudication terms. Transfer of the original claim/recovery rights prevents promising the same owner both reimbursement and free later recovery. Insurance cannot mint compensation or certify that a destroyed domain rejected.

8. An interstellar payment and local autonomy

Bob sends Alice an authenticated invoice with a B-issued coupon naming A. Alice may acquire her eventual RLD inputs later. She chooses accepted covers, fees and recovery policy, prepares x locally and retains its exact object. While the historical creation receipt is generated, A continues ordinary settlement. If a proof cannot be obtained within Alice's chosen local willingness to wait, she requests controller cancellation; time itself authorizes no release or refund.

Once the compliant terminal proof is ready, the controller commits Exported. Alice can go offline. Carriers move the capsule over a succession of real contacts. B verifies it and atomically imports once or permanently rejects. After import Bob can pay Carol locally without A. If B holds an onward coupon, it can prepare another export without a payment-specific returning permission.

```
Advance contact: B -> A: source-bound coupons; no future inputs
At A: Prepare -> independent controller escrow; native head continues
Controller: Exported(proof-ready) OR Canceled(local return once)
After Exported: carry exact capsule forward through actual contacts
At B: Imported(onward local use) OR Rejected(source recovery once)
```

Ships prepare accepted domains, local balances, coupons, prover/current-state custody, succession material and optional grants before isolation. A private key or copied ledger does not relocate a remote balance. A minority partition waits for its authority, while distinct prepared homes continue with distinct funds.

New information over distance d cannot arrive before d/c. Contact waiting, proving, serialization, receiving verification and consensus add delay. Prepositioned RLD makes local purchases possible; it does not accelerate a new distant payment. Goods delivery, lending, exchange and liquidity are separate agreements.

9. Provenance and the spread of trust

A short execution proof does not erase a historical authority dependency. Carrying every configuration/epoch label forever, however, makes even a single component eventually exceed finite receiving bounds. We therefore distinguish actual causal ancestry, which the execution relation verifies, from the acceptance envelope explicitly advertised to a payer and recipient.

An authority family F is a stable admitted domain identity plus immutable succession/custody constraints. Its accepted configurations, height keys and source-created controllers remain within F only when their complete authenticated transitions satisfy those constraints. Key rotation creates no new family merely because a new offset or epoch is used. An incompatible governance or security policy is a different family unless the receiving cover already explicitly permits that succession. A transport node is not a family dependency merely by carrying bytes.

A cover A is an immutable canonical sorted set of family descriptors with its required proof/key suites and incident rule. Its ID hashes that exact definition. A cover certificate is the monetary execution proof's statement that every actual ancestor of the amount satisfies A. It is not an attestation by a new risk oracle. Definitions and current descriptors remain available; recursively proved old history need not remain a live label vector. The default reference profile bounds a cover at 4096 family entries and an output at 64 nonzero cover components; these are finite resource choices, not performance results or global limits on domains.

Mint verifies its exact work slot and full placement ancestry against its signed initial cover. Transfer, preparation, controller decision, import and recovery require their executing families to be covered. Within unchanged A, the new proof verifies the previous coverage statement plus each new family membership; no archived set of every prior epoch is needed to prove the next use. A local ledger still keeps complete current outputs, cover definitions, terminal maps and access data.

```
Covered(value,A): verified ancestry uses only families allowed by A
Step under family F: require F satisfies A; preserve coverage A
Lift(A,B): require explicit owner authorization AND A subset B
Allocate: conserve each consumed A exactly before any signed Lift
Coalesce: add amounts only under the SAME resulting cover ID
```

Subset means inclusion of family rules AND no stronger claimed security than A allowed: B must accept every history A admitted. Native verification compares the complete bounded canonical definitions or an already verified exact definition witness; a hash match alone is insufficient. An immutable cover cannot silently replace its members. A new cover version is a new ID. Existing coupons retain their original accepted IDs.

Owner-authorized Lift deliberately broadens the advertised risk envelope. It does not remove a real dependency or turn risky history safe. For example, distinct configurations of one admitted family can circulate indefinitely under a fixed A without accumulating per-epoch tags. Two different actual paths can share A only because the owner and receiver explicitly accept that common envelope. A wallet must show this conservative broadening, preserve unrelated components/change and never perform it as an invisible convenience.

A recipient whose coupon accepts A checks the coverage proof, not an arbitrarily chosen ancestry leaf. Onward use under A or an explicitly accepted superset needs current state and the previous certified proof, not old Mint-origin lists. Restricting from A to a narrower/noncomparable B cannot follow from a coverage claim alone. It requires retained detailed ancestry witnesses and a fresh full proof, or an exchange of existing balances with another willing owner. If those witnesses or willing counterparties do not exist, narrowing refuses; no new money or risk erasure is invented.

A conservative incident concerning any family inside A can quarantine the entire A-covered portion, including funds whose undisclosed detailed path did not use that family. This is the cost of bounded current attribution. Exact narrow covers reduce that exposure at an interoperability cost. Accepted policy cannot grow to include infinitely many unrelated families inside a fixed-size encoding; a new finite cover needs explicit acceptance. Arbitrary new committees are not universally trusted. The construction supplies sustained circulation within an agreed bounded cover, not lossless constant-size description of every possible global risk set.

Thus RLD remains the only currency, while acceptance and market value may differ across cover classes. Shared covers are an interoperability agreement, not guaranteed fungibility. Incidents travel causally; unseen valid quorum forks remain undetectable from silence. Funded insurance and exposure limits cannot make a distant compromise disappear.

10. Transport and durable evidence

The carriage model is store-carry-forward over actual contacts, compatible with the delay/disruption model of Bundle Protocol version 7 [4]. A transport envelope binds its version, currency/application domain, whole content hash, length, origin and destination. Fragments bind positions and exact whole-object commitment. A receiver reconstructs and authenticates the complete object before monetary processing. Transport expiry can remove an attempt, never a monetary obligation.

An admitted custodian durably stores complete agreed bytes before acknowledging custody. A digest without the bytes is not evidence retention. Capacity refusal precedes a new obligation. Custody contracts bind retention scope, replication, failure reporting, service fees and renewal responsibility. Independent copies reduce single-carrier loss but not correlated failure or fraud. A signed custody receipt proves an assertion of custody, not successful future delivery or destination settlement.

End-to-end authentication persists across route changes; link authentication secures the present contact [5]. Two-way contacts may negotiate missing dependencies. A one-way carriage package contains the receipt, QC and all current material needed by its declared verifier and next monetary execution, or relies on an explicitly arranged later material delivery. The latter remains pending until complete. Candidate advertisements cannot change a ticket's destination or supply a trusted domain root.

A request identifier, content commitment and durable receiver record make retries idempotent. Receipt loss resumes carriage without another debit. Ledger acceptance, byte retention, relay compensation and optional receipt return are separate events. No forwarding schedule, satellite range or unavailable physical route is invented by monetary consensus.

10.1. Neighbor discovery and carriage scheduling

A normally operating network node runs bounded discovery and carriage alongside its ledger. It begins with a configured neighbor, an observed physical contact or an authenticated carried introduction, then exchanges signed transport identities, supported capsule/fragment profiles and bounded route hints. An advertisement binds its sender identity/key, application/network tag, sequence and advertised contact information. A verified advertisement supplies a candidate contact, not a currency descriptor, validator admission, proof of current reachability or payment credit. No mandatory Earth directory participates in monetary validation.

Nodes authenticate an actual contact under its configured physical/link profile, negotiate supported byte/custody bounds when two-way communication exists, and retain admitted whole-object commitments with resumable fragment positions. Neighbor observations and route hints expire only as local scheduling information. They cannot change a ticket, erase an obligation or authorize a refund. One-way opportunities use prearranged supported profiles and whole-object authentication; a receiver cannot invent an interactive challenge response that never occurred.

Local scheduling chooses among available contacts using observed capacity, destination hints, retained custody and configured replication budgets. Fair bounded queues include incident evidence, returned terminal decisions and data completion, so new traffic cannot automatically evict those classes. Duplicate whole-object commitments share retained immutable bytes; acknowledgments follow durable custody, not receipt of a digest. Losing an acknowledgment resumes the same transfer without another monetary debit. Routing is best effort over actual opportunities: progress requires a physical forward path and funded custodians, not an asserted global shortest route.

11. Fees, liquidity and incentives

Every owner-initiated local payment and cross-domain payment has an explicit positive RLD service fee and a positive work-author fee of at least one atomic unit, funded from authorized existing RLD. Preparation service charges, cancellation fees and post-release recovery charges are separately bound; cancellation cannot retroactively reclaim already performed preparation service. Its signed allocation specifies consensus operation, monetary verification and a positive archival-reserve contribution; a sponsor may pay on an owner's behalf through exact funded inputs. The work-author share names a verifiable prior Mint event and its exact payout policy; the native program verifies that binding and the signed positive allocation as a transfer of existing RLD. A cross-domain ticket binds its source/destination service allocations before preparation, including the required work-author share; import cannot bypass it. Mint and nonmonetary grant placement are not fee-paying purchases, so first issuance has no prior-balance prerequisite. No label such as miner fee authorizes an unbound future deduction. Mint itself requires no RLD admission fee, so the first successful miner can obtain a balance without a circular funding prerequisite.

A destination capability family is issued only against a funded service agreement or sponsor budget covering its stated bounds. A zero-balance recipient may have a sponsor; that is a real payment by another party, not free perpetual capacity. Source export pays its exact source services; import credits the recipient and the bound destination services; recovery pays only its original formula. All sums, controller-service budgets and cover assignments are owner-authorized. New fees cannot retroactively modify an exported ticket.

An archival reserve is existing RLD locked under an explicit threshold-controlled service policy. Records bind their remaining obligations, funded capacity and custodian set. Scheduled releases pay documented service under that policy; changing custodians requires authenticated handoff of complete data and liabilities. New capability families stop when funded capacity is exhausted, while existing terminal decisions and liabilities remain retained. A reserve cannot be withdrawn as surplus while its covered obligations persist. Without an independently specified proof-of-storage construction, releases still trust the agreed service authority; a receipt alone proves neither storage nor indefinite solvency.

Relay contracts are finite purchased services, also in RLD. Prepayment exposes a customer to nonperformance; delayed payment exposes a carrier to credit and return-path risk. Funded escrow, reputation, collateral and disclosed adjudication may distribute these risks. An arbitrary one-way message is not an automatically fair exchange of delivery and payment. No new subsidy is created by uptime, receipts, self-payments or newly created identities.

Mining rewards distribute the fixed budget and do not prove that validators, carriers or archives can operate profitably. After issuance ends, they rely on service fees, voluntary support or other disclosed funding. Outstanding capability liabilities must not assume future inflation. No finite reserve buys mathematically infinite storage; continued service needs funded renewal and succession. Failure of that economic premise can stop progress without granting a refund or rewriting the cap.

Long journeys also lock capital. Under stable-flow conditions, an export flow lambda and average unavailable interval W imply approximately lambda W of in-flight value [6]. Prepositioned RLD, reciprocal flows and separately funded liquidity can improve usability. Lending, insurance, external-currency exchange and delivery of goods remain additional economic arrangements. The basic payment operation is not atomic exchange of arbitrary distant assets or goods.

12. Conservation and safety argument

Consider a consistent causal cut including every source/controller ancestor of an included decision. U counts ordinary outputs and already issued balances held in preparation/service/insurance escrow or an unredeemed cancellation claim, once. F counts released cross-domain principal plus destination fees not yet imported or recovered. Z is owner-authorized retirement, I finalized Mint, and C the fixed cap. Coupons, proofs, keys and byte copies carry no monetary amount.

```
U + F = I - Z
I + B + Q = C
U + F + B + Q + Z = C
C = 4096 * 1000000 * 0.005126953125 RLD
```

B is remaining active unissued capacity; Q is detached capacity in prepared controllers, released tickets or unredeemed cancellation/rejection return claims. A detached slot is counted once in Q, never also active in B. Destination installation or exact source return moves it back to B. Lost objects remain unavailable in their last allocation, not newly minted elsewhere.

Lemma 1: authority exclusion. PBFT quorums intersect in f+1 identities among 3f+1. With at most f Byzantine signers, durable locks and correct view changes exclude conflicting native commits and conflicting terminal decisions within each controller. The source assigns a coupon and inputs to only one controller; a controller can neither release and cancel nor change its birth statement. Quorum counting without the lock/custody premises is insufficient.

Lemma 2: verified value. Native execution starts with zero issued balance and fixed slot space. It checks every owner, Mint, partition, cover step, fee, delegation and redemption. The controller relation derives its immutable amount or slots solely from valid native creation. With correct program implementation and recursive soundness, certified proofs establish those same statements without cold full replay. No unchecked host, committee-signed balance or conditional proof can introduce value. This validity statement does not establish absence of another valid branch after threshold compromise.

Lemma 3: cancellation versus release. A native preparation removes ordinary spending rights and delegates its exact locked object. Native commands cannot reclaim it without its Canceled decision. A Canceled QC excludes an Exported QC by Lemma 1, and RedeemCancel records redemption once. An Exported proof alone without QC grants no destination authority. Thus pre-release cancellation and destination credit cannot both consume one honest-controller escrow.

Lemma 4: import versus recovery. For an Exported object, destination J[x] and R[p] become Imported or Rejected, never both. Source rejection recovery checks valid creation, release and permanent destination rejection, then consumes its own redemption map once. Exact copies, lost receipts, compaction and configuration changes do not reset these namespaces. No other destination matches the ticket. Therefore one released amount cannot produce both destination spendability and source recovery.

Proposition: conservation. Preparation replaces source outputs with change, funded service fees and exactly equal controller escrow inside U. Canceled/redeemed preparation redistributes U, including only agreed fees. Exported moves a from U to F. Import moves a from F to recipient and destination-service outputs in U. Rejection changes neither; recovery moves a back, partitioned as (a-r)+r. Mint increases U and I by rho; retirement reduces U and increases Z. Lemmas 1-4 exclude duplicate consumption. Induction preserves the equations.

Proposition: issuance cap. Each root's slots are partitioned into consumed, active and detached identities. Mint consumes one active next slot. PrepareClaim detaches an exact unconsumed range and preserves disjoint residuals. Cancellation, installation and rejection return relocate precisely that detached range once, without restoring its old grant. No transition introduces a new (s,j). Hence I+B+Q=C and 0 <= I <= C under valid exclusive histories. Cover lifting, controllers and new domains never enlarge C.

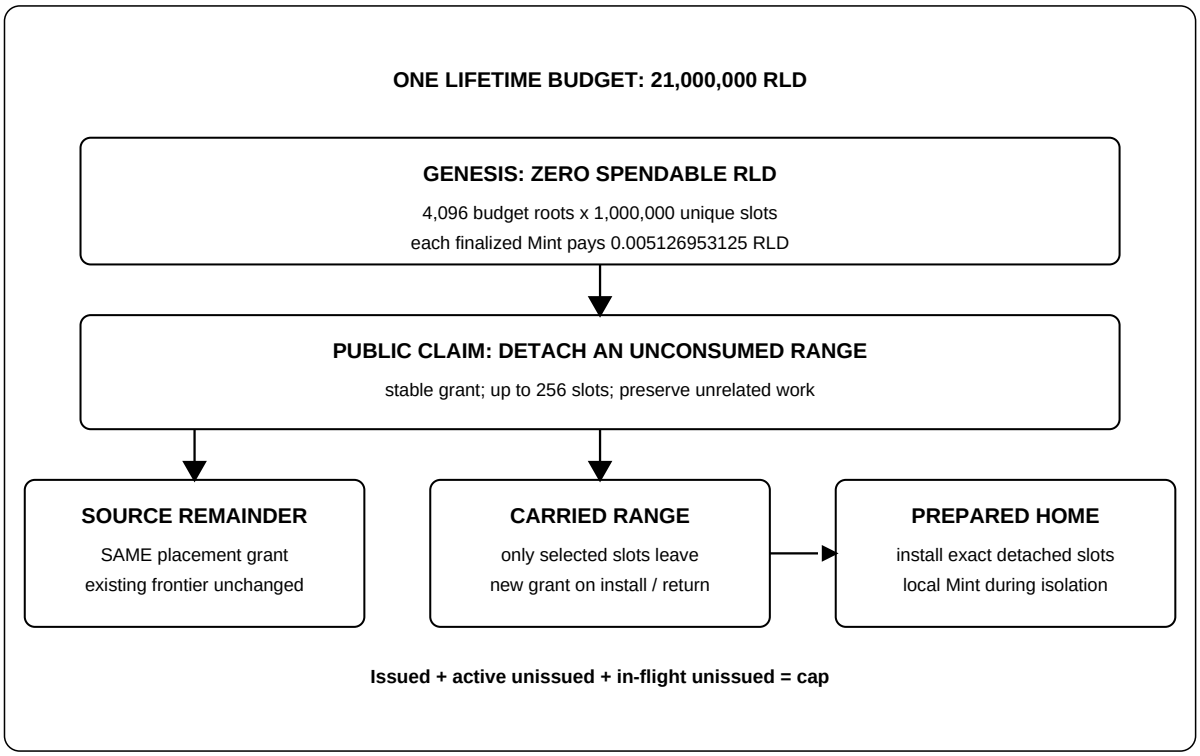


Figure 2. A placement grant survives splits of unrelated source slots. Only selected slots leave its authority; installation or return gives those slots a new grant. Mint and range transfers preserve the fixed issuance budget.

Proposition: independent preparation and proving. Let two valid claims select disjoint unconsumed ranges of the same G. Removing either range leaves the other covered by a retained G segment, with unchanged placement-work inputs. Thus either serialized order preserves the other's allocation eligibility. Actual overlap, Mint of a selected slot or moving those slots changes eligibility legitimately. A receipt for committed H_p remains valid under later native extension; a controller transition depends on I_x rather than the latest native head. Native settlement therefore need not pause for historical proving.

Proposition: bounded-cover continuation. Suppose every actual ancestor satisfies immutable A and the next executing family is accepted by A. The recursive step verifies the previous coverage claim and that family membership, so the result remains Covered(A). Its current cover representation does not grow with history length or key rotations. Authorized A-to-B lifting follows inclusion and conserves amounts. It broadens conservative acceptance; it supplies no narrowing claim or incident immunity.

Proposition: conditional progress. Under responsive native/controller quorums, bounded fairly admitted work, retained witnesses and a functioning pinned prover, preparation eventually yields release or an authorized cancellation. Released conforming material eventually imports if a physical forward path exists, original cryptographic/service assumptions still hold and rejection has not won. Recovery needs an authentic rejection path back. Current local payments use only their own home and available resources. No unconditional deadline follows when these premises fail.

Information boundary. The same observed proof cannot reveal an unseen conflicting valid certificate. Eliminating all remote ordering assumptions while allowing arbitrary disconnected use of the same funds would require another conflict-resolution construction. This paper retains exclusive homes and explicit accepted authorities; it does not use risk covers, work or recursion to claim away that limit.

13. Configuration, keys and long journeys

A validator handoff is a special committed command. It binds old/new configurations, exact activation boundary, predecessor, complete monetary state, complete cover tables and definitions, imports, rejections, capability-use/closure maps, outstanding reservations, service liabilities, placement grants, segment frontiers and anchors. The old configuration commits it; a new quorum authenticates and durably installs the fully execution-verified current state and certified receipts before acting. Old native height keys cannot authorize the old configuration beyond that boundary. Outstanding controllers retain their immutable original keys and authority until their own terminal decision; a native handoff neither transfers nor cancels their escrows. Copying balances alone is not succession. Without an authorized old handoff, an administrator cannot resume stopped money under a replacement root.

For native ordering certificates we describe a finite per-height key schedule using ordinary Ed25519 [7], SHA256 commitments and HKDF-SHA256 [8]. It is a construction requiring secure erasure and custody analysis, not a claim of a newly standardized forward-secure signature. Each validator's epoch descriptor commits to a Merkle root of K = 65536 per-height public keys. Leaves bind C_RLD, domain, validator, epoch, offset and public key; internal nodes use a distinct tag. The epoch fixes the first sequence position; the offset cannot be chosen by the sender.

For initial random secret S_0, define S_(i+1) = SHA256(enc("RLD-KEY-NEXT", binding, i, S_i)). Derive the Ed25519 seed for i with HKDF-SHA256 from S_i, using the epoch binding as salt and a distinct key-purpose information string. The public tree is prepared before epoch activation. The private signer retains only its current S_i, exact locks and response journal, never a recoverable copy of retired S_i or the initial seed. Each PBFT signature carries its exact offset and Merkle authentication path. Signature inputs include phase, view, sequence, predecessor and body, so a path cannot rebind a vote.

After a position is irreversibly committed, the signer durably retains the commit, lock/head and exact already signed responses, advances to S_(i+1), and erases S_i and derived private material before signing a later position. Retransmission uses retained bytes, not regenerated retired signatures. Current-secret compromise should not recover earlier secrets under the hash/KDF assumptions; it can compromise future keys until a valid succession. Consensus safety still needs the active fault bound, secure heads, prevention of concurrent cloned signers and verified erasure. Ordinary disk deletion, encrypted old backups or a local timestamp do not prove those conditions.

The key schedule has finite capacity. A valid old/new handoff must activate a fresh independently generated tree before exhaustion; otherwise signing stops. Epoch identities never reset a source, capability or import namespace. Current key descriptors and authentication paths travel with relevant material; historical handoff validity is folded into the execution receipt, while audit archives retain the history. The erasure assumption narrows later key-theft risk; it does not stop an already compromised quorum from signing conflicting present histories, or signatures being broken by later cryptanalysis. Forward-secure research makes the distinction explicit [9].

Controller keys use a separate finite Merkle pool of 65536 independently generated Ed25519 keys per admitted validator and pool epoch. The native configuration commits each pool root only after admission validates the complete bounded public inventory, distinct member keys and exact epoch/purpose binding. Operators retain that public inventory to supply paths; ordinary payment capsules carry selected leaves, not every future key. Native height and controller pools use separate randomness/purpose domains. Incorrect pool data refuses before activation. Prepare consumes the next common pool index exactly once and includes the ordered validator public leaves/paths; no fresh unanimous online acknowledgment is needed to create a controller. Its descriptor pins the original n validators, threshold, pool epoch and index. Keys are independent of native height seeds and cannot be used as ordinary votes. Controller state retains its own PBFT locks across native progress and reconfiguration. After terminal commitment it retains exact responses and erases the corresponding private key. Pool exhaustion refuses new preparations until an authenticated new pool is active; it cannot stop an existing controller's cancellation/release. Old operators retain outstanding keys/custody obligations after native handoff, or those objects can lose availability. A new quorum cannot seize them.

Every journey declares supported cryptographic suites and an acceptance horizon. Unknown or cryptographically unsound evidence refuses or remains pending. Post-quantum standards can inform an explicitly instantiated successor

[10], but a new algorithm name does not renew all old history. Revocation warnings propagate causally. Secure long-term operation requires generational maintenance and preserved anchors, not a promise of eternal signatures.

Owners choose recovery policies before loss, such as independent threshold keys. Recovery requires an ordinary certified command at the current home; a relay or founder cannot reset ownership. Backups must reconstruct current certified state, permanent decisions and freshness anchors before signing or voting. A restored old image cannot declare itself current. Loss of all state and independent anchors is not repairable by a protocol assertion.

The transparent baseline exposes amounts, keys, advertised acceptance covers and some timing and route metadata. Encrypted carriage does not hide all linkage. Privacy systems such as Zef [11] address a different problem; integration needs a proof that hidden amounts, acceptance predicates and duplicate prevention remain sound.

14. Capacity, archives and failure bounds

Distinct homes can add capacity because their balances are distinct. Native settlement, admitted work, controller decisions, proving and receiving are separate queues. A bounded positive service share prevents one admitted class monopolizing others under the declared workload. A hot destination or shared prover is still a bottleneck. One hundred million completions per day is about 1157.41 per second; this arithmetic is not measured capacity. Owner validation, QC, inclusion and durable usable data count toward completion.

Preparation commits a fixed historical checkpoint; proof workers continue from retained exact witnesses while the native ledger advances. Per-object controllers isolate terminal proving from ordinary settlement. Each consumes bounded keys, escrow, response journals and custody. Native admission refuses before preparation when its funded outstanding-controller limit is reached, and the reference profile permits at most 4096 unresolved controllers per source. An already prepared object's owner can request cancellation without waiting for a departure proof, provided its original controller and source remain available. Cancellation/recovery queues retain their declared service shares; new preparations cannot evict those obligations.

A cover has at most 4096 family entries; an output has at most 64 nonzero components. The reference interface limits an operation to 64 inputs and 64 monetary outputs and current capsule material to 16 MiB, with separately bounded receipts, signatures and nested objects in Appendix A. These are refusal bounds, not claims of ordinary-device performance. A payer may split too many components across funded operations. Splitting does not make a nonaccepted cover accepted or justify risk narrowing. Long circulation under a fixed agreed cover adds proof work, not a growing per-coin epoch vector.

A capsule carries every declared current item or identifies a previously independently retained exact definition allowed by the coupon. A cold one-way receiver cannot be assumed to have that cache. Missing material remains pending. Current UTXOs, permanent maps, cover definitions, source creation statements and controller response data remain available to the next prover. Digest agreement alone gives no update witness, custody or monetary right.

Permanent duplicate-prevention state grows with use. At 100 million 64-byte records daily, raw growth is about 2.34 terabytes yearly before indexes and replicas. Compact roots and recursive receipts do not delete source delegations, consumed coupon leaves, terminal decisions or redemption identities. Available authenticated storage/witness services and funded archival succession distribute this obligation. Global state is not bounded merely because an individual proof is small.

Service reserves pay finite, explicitly contracted capacity. Before issuing a coupon family, the destination records a named renewal payer, reserve, custodian set and funded operating horizon. Authorized renewals add existing RLD and extend service; no remote timestamp expires monetary rights. If renewal stops, new coupon issuance and new unsupported preparations stop. Existing obligations enter a retained pending/service-deficit state and seek funded custody handoff. Capacity or insolvency alone cannot certify rejection, import or refund. The protocol promises safety while paused, not service without resources forever.

Historical audit archives support forensics, reproof and cryptographic renewal under explicit retrieval/replication contracts. Normal same-cover continuation uses current state and certified predecessor receipts, not a transcript of every old Mint. Narrowing an acceptance envelope may require detailed ancestry unavailable from that summary. Old-suite

history cannot be renewed after its security has already failed by simply proving that old signatures once verified. Suite succession must occur while predecessor assumptions remain credible and preserve all state and decisions.

Independent replicas, prepared key renewal and retained current data reduce failures but do not establish physical delivery, secure erasure or perpetual operation. Proving cycles, native latency, worst-case capacity, fee adequacy and independent operation require actual measurements. The finite construction is conditional on correct implementations and maintained resources; a body revision does not produce those facts.

15. Relation to prior work and scope of the contribution

Bitcoin demonstrates open-work selection of a monetary history [1]. PBFT supplies local state-machine replication under explicit validator assumptions [2]. This design keeps conflict resolution local to avoid a distant branch contest over an isolated community's completed payments; it changes the trust model rather than improving every property at once.

FastPay and Zef show that payment workloads can avoid a single global total order [11,12]. Asynchronous asset-transfer research separates transfer from general consensus [13]. Escrow and bounded-counter techniques partition finite rights before disconnected use [14]. Those database results do not supply Byzantine monetary security. Here work entitlements additionally require authenticated exclusive histories, valid execution and permanent import/placement decisions.

IBC separates remote consensus verification, transport and application transitions [15]. Cross-shard work studies replay and stale-message failures [16]. Delay-tolerant networking supplies the appropriate carriage model [4,5]. Recursive zkVM composition provides a concrete route to execution receipts [17,18]; it does not supply RLD monetary semantics automatically. We do not claim a new BFT algorithm, proof primitive, general transfer concept or invention of store-carry-forward networking.

The contribution proposed here is their explicit composition for one RLD currency across extremely delayed contacts: independently verified monetary execution, advance source-bound capabilities, cancellable local preparation, independent terminal controllers, rejection-only recovery after release, explicit cover-class amounts, stable-grant divisible work entitlements and finite proof/key/archive succession. Its novelty and security require independent comparison and analysis. Interstellar terminology alone establishes neither.

The most immediate applications are already-separated communities, moving platforms and disrupted infrastructure with real custody and payment needs. Whether users prefer RLD to existing money depends on practical costs, acceptance, access and risk. A payment mechanism does not itself create demand, stable purchasing power or social impact. Physical links, sustainable institutions and useful adoption remain necessary.

16. Conclusion

Interstellar payment becomes possible by placing exclusive authority before disconnection and carrying certified changes through actual contacts. A destination supplies bounded source-specific capacity before the payer knows future inputs. Native preparation delegates an exact escrow; independent one-decision controllers separate proof generation from ordinary settlement and offer cancellation before release. After release, only destination import or permanent rejection can decide the ticket, and only rejection permits source recovery. No receipt must return before onward local use.

runlai (RLD) is the sole native payment and fee currency, capped at twenty-one million with zero initial allocation. Stable placement grants preserve disjoint claims through unrelated source splits, while exact slots and public work conserve issuance. Owner-accepted finite covers support continued circulation without an ever-growing epoch vector; they deliberately broaden possible exposure and may quarantine more funds during an incident. They never erase real history or create another unit of money.

The construction specifies state, encodings, authorization, terminal exclusions, recursive relations and resource/failure behavior. Its claims remain conditional on accepted PBFT authorities, correctly implemented proof programs, available state, finite keys, funded succession and physical paths. These premises define an implementable research target and its limits; useful adoption and social impact require functioning systems and willing users.

Appendix A. Canonical objects and transition relation

Enc uses deterministic CBOR [19] with fixed-order arrays, definite lengths and shortest unsigned encodings. Monetary amounts are checked unsigned 128-bit integers; values above the uint64 CBOR range use tag 2 with the shortest nonzero big-endian byte string. Negative values, floats, maps, text normalization, duplicate IDs, trailing bytes, unknown tags/fields and integer overflow refuse. Text identifiers are fixed ASCII tags; IDs and SHA256 roots are 32-byte strings; signatures and public keys have their suite-fixed lengths. Lists declared as sets are strictly byte-sorted and duplicate-free. Wire and signature bytes must round-trip identically.

The protocol_id is RLD-PAYMENT-CTRL-1, encoding_id is RLD-CBOR-1, and native/controller relations have distinct profile tags RLD-NATIVE-RELATION-1 and RLD-CONTROLLER-RELATION-1. Every top-level signed/hashable protocol statement except the currency identity preimage begins with [type,protocol_id,C_RLD]. The currency preimage begins with [RLD-CURRENCY,protocol_id] and its ordered policy fields, excluding its own digest and later currency-derived state/key roots. All remaining fields follow the orders below. Stable controller/domain identities are chosen before later creation attestations. Operation IDs hash the complete command body with resulting object IDs, state roots, proof bytes and QCs excluded; output IDs then derive from that operation and index. In contrast, checkpoint/terminal statement IDs hash their COMPLETE canonical headers, including resulting state/material roots, excluding only proof/signature wrappers. A predecessor checkpoint ID is that header digest, independent of which valid QC subset encodes its certificate. Internal Merkle/work/key expansions use their dedicated tagged preimages and the same currency/profile binding. Outputs are addressed by finalized operation identity and output index. Signing an operation binds its complete canonical body, including every input, output, cover lift and fee.

```
Currency = [C_policy_fields,D_0_identity,root_count,slots,reward,
            denomination,work_suite,native_program,controller_program,
            verifier_parameters,encoding_id,limits,succession_rule]
Output = [output_id,owner_policy,home,sorted([cover_id,amount])]
FamilyRule = [family_id,initial_descriptor_digest,allowed_succession_profiles,
              allowed_signature_profiles,required_custody_profile]
Cover = [family_rules_sorted,proof_suite_rules,custody_rules,
         conservative_incident_rule]
Grant = [root,original_lo,original_hi,home,creation_body,cover_id]
Segment = [grant_id,lo,hi,cursor,mint_anchor]
Coupon = [family_id,index,source,destination,recipient_policy,kind,
          max_principal_or_slots,root_rules,accepted_covers,
          use_policy,fee_formulas,recovery_rule,suites,limits,funding]
Ticket = [x,p,source,destination,creator_policy,kind,inputs_or_range,recipient,
          principal,destination_fee,source_fees,change,cover_allocations,
          signed_lifts,cancel_policy,recovery_policy,nonce,versions]
Controller = [x,source,ticket_digest,coupon_digest,
              exact_escrow,configuration,pool_epoch,pool_index,
              ordered_keys_and_paths,material_commitment,limits]
Terminal = [x,initial_statement_digest,source_checkpoint_id,
            Exported_or_Canceled,
            exact_ticket_or_return_digest,material_root]
Checkpoint = [profile,domain,sequence,predecessor_certificate,
              old_root,new_root,batch_digest,configuration,material_root]
```

Owner policy in the reference profile is threshold Ed25519 over at most 16 distinct byte-sorted keys. The creator policy also signs preparation. Every actual input must satisfy its own threshold; signatures cannot stand in for another owner's approval. Validators are distinct admitted identities, n = 3f+1 <= 127; QCs contain at least 2f+1 distinct ordered votes for the exact statement/view/position under the descriptor. PBFT prepare, commit and view-change domains are separate. Native and controller signature-purpose domains are separate even when operator identities coincide.

The initial material commitment hashes the ticket, coupon terms, cover definitions and output/current data only; later source-creation and terminal headers, receipts and QCs remain external certifications and are excluded. Controller I_x contains no digest of its enclosing source checkpoint or preparation operation; source creation is bound externally by receipt/QC and inclusion after that checkpoint exists. Its later Terminal journal binds that exact source checkpoint. The x identity hashes the tagged C_RLD/source/creator-policy/nonce statement before the final ticket is formed.

State maps use ordered Merkle commitments. Each collection tag binds currency, home and map name. A leaf hashes its collection tag, exact key and canonical value; internal nodes hash an internal tag and ordered child hashes. For n>1 leaves the tree splits at the largest power of two strictly below n; a singleton is its leaf hash, and an empty tree has a distinct tagged hash. The collection root additionally binds its tag and leaf count. Membership carries the exact leaf/index/count and sibling positions; absence authenticates adjacent ordered neighbors or exact endpoints. Native proof execution authenticates every touched old value against the predecessor root and computes the resulting root; an unbound caller-supplied snapshot is not state. Total counters and pool indices are committed along with the maps.

FamilyRule profile lists are finite sorted sets of approved profile IDs, not arbitrary host predicates. Cover lifting requires the same initial family anchor, inclusion of every old allowed succession/signature profile, the identical required custody profile and unchanged conservative incident semantics. Different custody rules require detailed reproof rather than an assumed ordering of security. Nonces are 32 bytes; sequences, views and allocation counters are checked uint64 values that refuse on exhaustion. Root/slot indices are checked against the fixed monetary ranges.

The reference envelope permits at most 64 inputs, 64 outputs, 64 cover components per output, 4096 families per cover, 127 validators, 16 owner keys per policy, a 1 MiB execution receipt, 16 MiB total current capsule bytes, nesting depth 32 and 4096 distinct carried objects. Counts and aggregate bytes are checked before allocation; expanded current material shares the same limits, while historical witness computation is separately metered by the proving service. A coupon may set lower limits, never silently raise the currency limits. These maxima select a finite interface, not a benchmark or a security parameter for an unimplemented guest.

Native operations are LocalTransfer, Mint, PreparePayment, PrepareClaim, CloseUnusedCoupon, RedeemCancel, ImportPayment, InstallGrant, RejectExport, RedeemRejected, FundService, RenewService and Handoff. Each first verifies exact version/currency/profile, canonical input, signatures, old-state membership and applicable resource bounds. It computes all effects before committing any map or counter. Unsupported operations refuse without mutation.

LocalTransfer consumes actual local UTXOs, conserves each cover allocation and applies only signed inclusion-proved lifts before creating outputs and explicit positive fees. Mint follows Section 3.3 and records the unique consumed slot. Preparation consumes ordinary inputs or selected active slots, reserves funded services/key index, creates one immutable Controller and sets the unused S[p] to Delegated(x). Duplicate x/p/body discrepancies refuse; exact already committed commands return their retained result without re-execution.

CloseUnusedCoupon requires the use policy and S[p] unused, then permanently marks it Closed. RedeemCancel checks retained creation, the exact legal Canceled transition/QC, no prior cancellation redemption and no known conflicting terminal outcome; it records redemption, restores the bound return and marks p Closed. Import/Install check valid controller Exported and destination R[p]/J[x] exclusion before credit. RejectExport requires valid release and recipient/recovery authorization, then consumes the same reserved leaf with permanent rejection. RedeemRejected checks valid release and certified destination rejection, original source binding and unused recovery map before credit. A command that has already redeemed cannot be revived by a late observation or another certificate encoding.

FundService and RenewService transfer already issued owner-authorized RLD to explicit escrows and update only their bound capacity liabilities. No service receipt mints money or signs a monetary rejection. Handoff verifies the old committed authority, new acknowledgment and complete native state/custody before activation; controller descriptors, grants, coupons and redemption namespaces persist. Native signing stops if no authorized next configuration/key pool exists. These rules and the native/controller proof recurrences specify the relation; executable artifacts and their image commitments must realize it before monetary use.

References

[1] S. Nakamoto. Bitcoin: A Peer-to-Peer Electronic Cash System, 2008. https://bitcoin.org/bitcoin.pdf

[2] M. Castro and B. Liskov. Practical Byzantine Fault Tolerance, OSDI, 1999. https://pdos.csail.mit.edu/6.824/papers/castro-practicalbft.pdf

[3] A. Biryukov, D. Dinu, D. Khovratovich and S. Josefsson. Argon2 Memory-Hard Function for Password Hashing and Proof-of-Work Applications, RFC 9106, 2021. https://www.rfc-editor.org/rfc/rfc9106.html

[4] S. Burleigh, K. Fall and E. Birrane. Bundle Protocol Version 7, RFC 9171, 2022. https://www.rfc-editor.org/rfc/rfc9171.html

[5] E. Birrane and K. McKeever. Bundle Protocol Security (BPSec), RFC 9172, 2022. https://www.rfc-editor.org/rfc/rfc9172.html

[6] J. D. C. Little. A Proof for the Queuing Formula: L = lambda W, Operations Research, 1961. https://doi.org/10.1287/opre.9.3.383

[7] S. Josefsson and I. Liusvaara. Edwards-Curve Digital Signature Algorithm (EdDSA), RFC 8032, 2017. https://www.rfc-editor.org/rfc/rfc8032.html

[8] H. Krawczyk and P. Eronen. HMAC-based Extract-and-Expand Key Derivation Function (HKDF), RFC 5869, 2010. https://www.rfc-editor.org/rfc/rfc5869.html

[9] M. Bellare and S. K. Miner. A Forward-Secure Digital Signature Scheme, CRYPTO, 1999. https://cseweb.ucsd.edu/~mihir/papers/fsig.pdf

[10] NIST. Module-Lattice-Based Digital Signature Standard, FIPS 204, 2024. https://doi.org/10.6028/NIST.FIPS.204

[11] M. Baudet, A. Sonnino, M. Kelkar and G. Danezis. Zef: Low-latency, Scalable, Private Payments, 2022. https://arxiv.org/abs/2201.05671

[12] M. Baudet, G. Danezis and A. Sonnino. FastPay: High-Performance Byzantine Fault Tolerant Settlement, 2020. https://arxiv.org/abs/2003.11506

[13] P. Kuznetsov, Y.-A. Pignolet, P. Ponomarev and A. Tonkikh. Permissionless and Asynchronous Asset Transfer, DISC, 2021. https://doi.org/10.4230/LIPIcs.DISC.2021.28

[14] V. Balegas, D. Serra, S. Duarte, C. Ferreira, R. Rodrigues, N. Preguica, M. Shapiro and M. Najafzadeh. Extending Eventually Consistent Cloud Databases for Enforcing Numeric Invariants, 2015. https://arxiv.org/abs/1503.09052

[15] Inter-Blockchain Communication. How the IBC Protocol Works. https://ibcprotocol.dev/how-ibc-works

[16] A. Sonnino, S. Bano, M. Al-Bassam and G. Danezis. Replay Attacks and Defenses Against Cross-shard Consensus in Sharded Distributed Ledgers, 2020. https://arxiv.org/abs/1901.11218

[17] RISC Zero. SuccinctReceipt, risc0-zkvm 3.0.6 API documentation. https://docs.rs/risc0-zkvm/3.0.6/risc0_zkvm/struct.SuccinctReceipt.html

[18] RISC Zero. Proof Composition, zkVM 3.0 documentation. https://dev.risczero.com/api/zkvm/composition

[19] C. Bormann and P. Hoffman. Concise Binary Object Representation (CBOR), RFC 8949, 2020. https://www.rfc-editor.org/rfc/rfc8949.html